

PROCESS SYNCHRONIZATION

PROCESS SYNCHRONIZATION

- **Types of Processes**
- **Cooperating Processes**
 - Share data/resources
 - Example: CPU, Memory, Files, Scanner
- **Independent Processes**
 - Do not share data

PROCESS SYNCHRONIZATION (Cont...)

Process P1 :

`x = shared;`

`x = x + 1;`

`shared = x;`

Process P1 :

`y = shared;`

`y = y - 1;`

`shared = y;`

If executed concurrently → **Race Condition occurs**

Producer Consumer Problem

```
int count = 0;
void producer( void )
{
    int itemP;
    while(1)
    {
        Produce_item(item P)
        while( count == n);
        buffer[ in ] = item P;
        in = (in + 1)mod n
        count = count + 1;
    }
}
```

Shared Variables:

```
int count = 0;
buffer[n];
in = 0;
out = 0;
```

Load Rp, m[count]
Increment Rp
Store m[count], Rp

```
int count;
void consumer(void)
{
    int itemC;
    while(1)
    {
        while( count == 0);
        itemC = buffer[ out ];
        out = ( out + 1 ) mod n;
        count = count - 1;
    }
}
```

Load Rc, m[count]
Decrement Rc
Store m[count], Rc

Producer I1,I2 | Concusmer I1,I2 | Producer I3 | Consumer I3

Printer Spooler Problem

- I1. LOAD Ri, M[IN]
- I2. STORE SD[Ri] "FILE_NAME"
- I3. INCR Ri
- I4. STORE M[IN], Ri

Spooler Directory	
0	F1.doc
1	F2.doc
2	F3.doc
3	
4	
5	
6	
7	
...	
...	

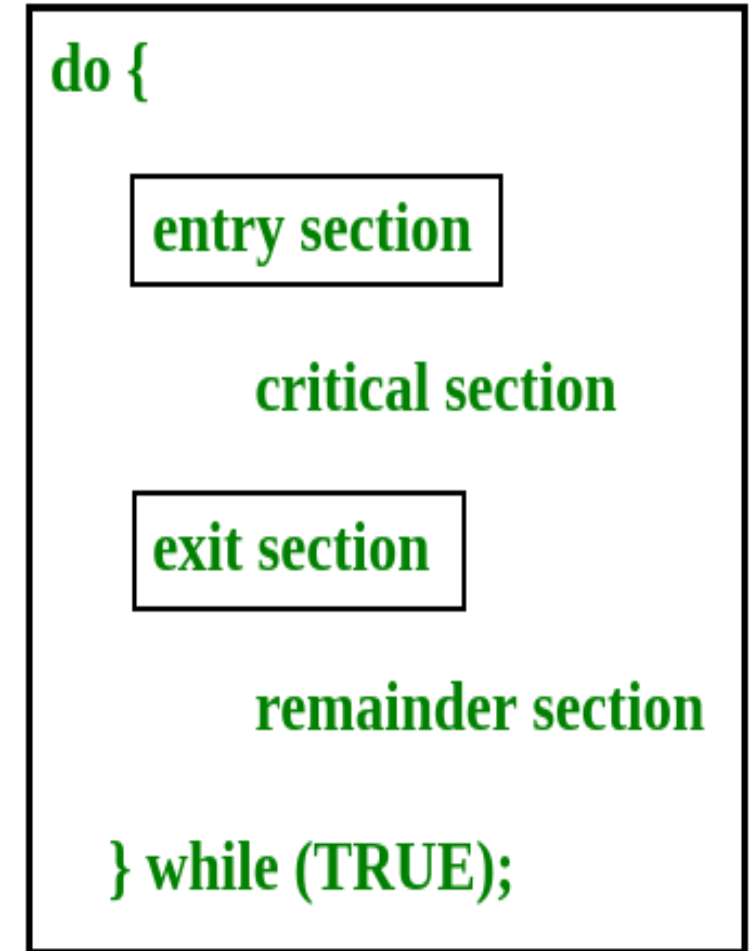
P1-> I1,I2,I3,* | P2-> I1,I2,I3.I4 | P1-> I4

Process Synchronization

Critical Section

Critical Section

- Critical Section is the part of a program where **shared resources are accessed**.
- Program has (1) Critical Section and (2) Non-Critical Section
- Critical Section is a place where shared variable, resources are shared.
- Each process consists of:
 1. Entry Section
 2. Critical Section
 3. Exit Section
 4. Remainder Section



Examples of critical sections in real-world applications

- A. Banking System (ATM or Online Banking)
- B. Ticket Booking System (Airlines, Movies, Trains)
- C. Print Spooler in a Networked Printer
- D. File Editing in Shared Documents (e.g., Google Docs, MS Word with shared access)

Requirements for Critical Section Solution

- Mutual Exclusion
- Progress
- Bounded Waiting
- No assumption about speed/number of processors

do {

entry section

critical section

exit section

remainder section

} while (TRUE);

Lock Variable Solution (Software Solution)

Code by Galvin

Acquire lock
Execute CS
Release lock

Sudo Code:

```
1. While(Lock==1);  
2. Lock=1  
3.   Critical Section  
4. Lock=0
```

Entry
Section

Exit Section

Lock=0 -> Vacant
Lock=1 -> Full (Not
Vacant)

Mutual Exclusion - Yes
Progress - No
Bounded Waiting - No

Hardware Solutions : Test and Set Instruction

- while(test_and_set(lock));
- Critical Section
- lock = false;

- boolean test_and_set(boolean *target){
- boolean rv = *target;
- *target = true;
- return rv;
- }

Lock Variable :

1. While(Lock==1);
2. Lock=1
3. Critical Section
4. Lock=0

Turn Variable (Also a Strict Alternation)

- Two processes share a variable called turn.
- `int turn;`
- If `turn = 0` → Process P0 can enter Critical Section
- If `turn = 1` → Process P1 can enter Critical Section
- Only the process whose turn it is can enter the Critical Section.

Process P0	Process P1
<pre>while (true) { while (turn != 0); // Critical Section turn = 1; // Give chance to P1 // Remainder Section }</pre>	<pre>while (true) { while (turn != 1); // Critical Section turn = 0; // Give chance to P0 // Remainder Section }</pre>

Mutual Exclusion - Yes
Progress - No
Bounded Waiting - Yes

Peterson's Algorithm

- Peterson's Algorithm is a software solution to the Critical Section Problem for two processes.
- It was proposed by Gary **Peterson** (1981) and is considered an improved version of the Turn Variable solution.

```
while (true) {  
  
    flag[i] = true;    // I want to enter  
    turn = j;          // Give priority to other process  
  
    while (flag[j] && turn == j);  
  
        // Critical Section  
  
    flag[i] = false;  
  
    // Remainder Section  
}
```

Mutual Exclusion - Yes
Progress - Yes
Bounded Waiting - Yes

Semaphore

- Semaphore is an integer variable which is used in mutual exclusive manner by various concurrent cooperative processes in order to achieve synchronization.
- A semaphore is an **integer variable** accessed only through two atomic operations:
 - wait() / down() / P()
 - signal() / up() / V()

□ Semaphore Types

1. Counting Semaphore ($-\infty$ to $+\infty$)
2. Binary Semaphore (0, 1)

Semaphore Code (Sudo Code) - Counting Semaphore

wait(S)

```
{  
  S.value = S.value - 1;  
  if (S.value < 0) {  
    add process to  
waiting queue;  
    block();  
  }  
}
```

signal(S)

```
{  
  S.value = S.value + 1;  
  if (S.value <= 0) {  
    remove process from  
waiting queue;  
    wakeup();  
  }  
}
```

```
do {  
  entry section  
  critical section  
  exit section  
  remainder section  
} while (TRUE);
```

Semaphore Code (Sudo Code) - Binary Semaphore

```
wait(S)
{
    if (S.value == 1)
        S.value = 0;
    else
        block();
}
```

```
signal(S)
{
    if (waiting queue empty)
        S = 1;
    else
        wakeup(process);
}
```

```
do {
    entry section
    critical section
    exit section
    remainder section
} while (TRUE);
```


Solution of Producer-Consumer Problem using semaphore

// Initialization

```
#define N 8
```

```
semaphore full = 0;  
semaphore empty = N;  
semaphore S = 1; //  
Binary 0 or 1  
int in = 0;  
int out = 0;  
item buffer[N];
```

//Producer Sudo Code

```
while (true) {  
  
    item p =  
    produce_item();  
  
    down(empty);  
    down(S);  
    buffer[in] = p;  
    in = (in + 1) % N;  
  
    up(S);  
    up(full);  
}
```

//Consumer Sudo Code

```
while (true) {  
  
    down(full);  
    down(S);  
  
    item c = buffer[out];  
    out = (out + 1) % N;  
    up(S);  
    up(empty);  
  
    consume_item(c);  
}
```

Solution of Reader-Writer Problem using semaphore

```
int rc = 0;
semaphore mutex = 1;
semaphore db = 1;

// mutex and db binary
=====
void Reader(void)
{
    while (true)
    {
        down(mutex);
        rc = rc + 1;

        if (rc == 1)
            down(db);
        up(mutex);
```

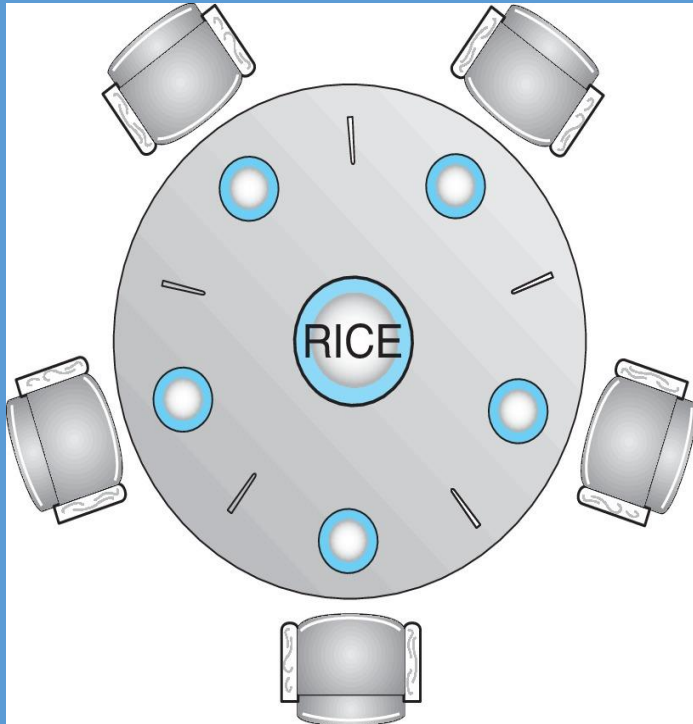
```
    read_data(); //DB

    down(mutex);
        rc = rc - 1;
        if (rc == 0)
            up(db);
        up(mutex);
        process_data();
    }
}
=====
```

```
void Writer(void)
{
    while (true)
    {
        down(db);
        write_data(); //DB

        up(db);
    }
}
```

Solution of Dining Philosophers Problem using semaphore



```
#define N 5
```

```
void Philosopher(int i)
```

```
{
```

```
    while (true)
```

```
    {
```

```
        Thinking();
```

```
        take_fork(i);
```

```
        take_fork((i + 1) % N
```

```
        EAT();
```

```
        put_fork(i)
```

```
        put_fork((i + 1) % N);
```

```
    }
```

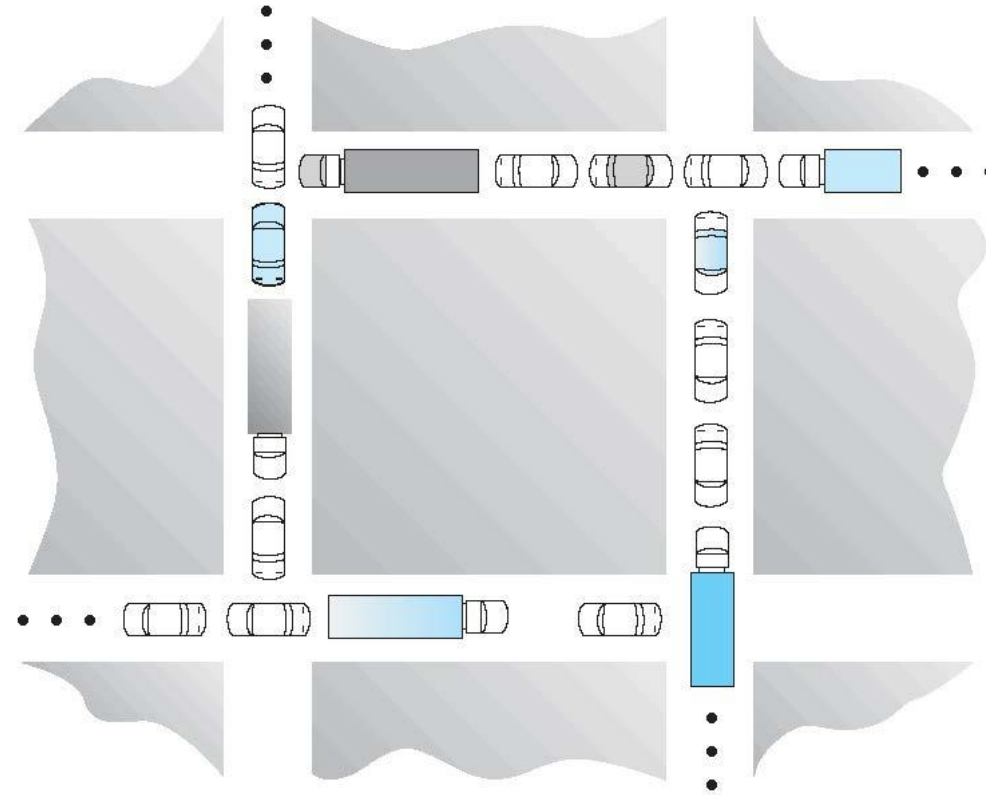
```
}
```

Philosopher	Left Fork	Right Fork
P0	S0	S1
P1	S1	S2
P2	S2	S3
P3	S3	S4
P4	S4	S0

Feature	POSIX	Java
Language	C / C++	Java
Thread Library	Pthreads	Java Thread API
Mutex Support	pthread_mutex	synchronized / Lock
Semaphore	POSIX semaphores	Java Semaphore class
Platform	Unix/Linux systems	Platform independent

Deadlocks

Deadlock Example with Cars



Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

- **Mutual exclusion:** only one process at a time can use a resource
- **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes
- **No preemption:** a resource can be released only voluntarily by the process holding it, after that process has completed its task
- **Circular wait:** there exists a set $\{P_0, P_1, \dots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1 , P_1 is waiting for a resource that is held by P_2 , ..., P_{n-1} is waiting for a resource that is held by P_n , and P_n is waiting for a resource that is held by P_0 .

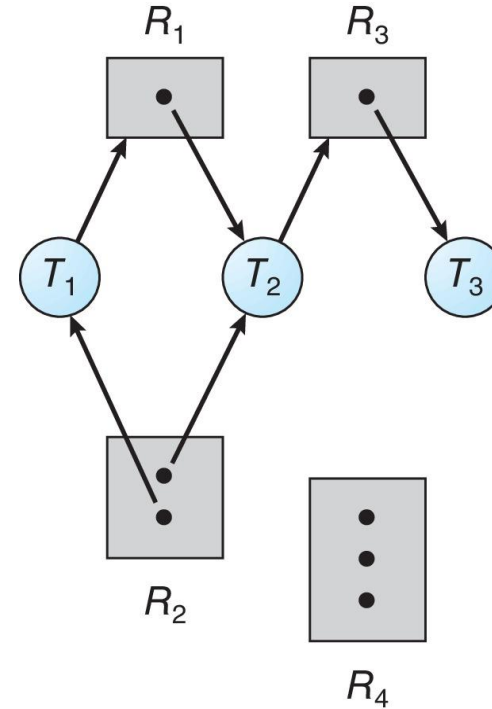
Resource-Allocation Graph

A set of vertices V and a set of edges E .

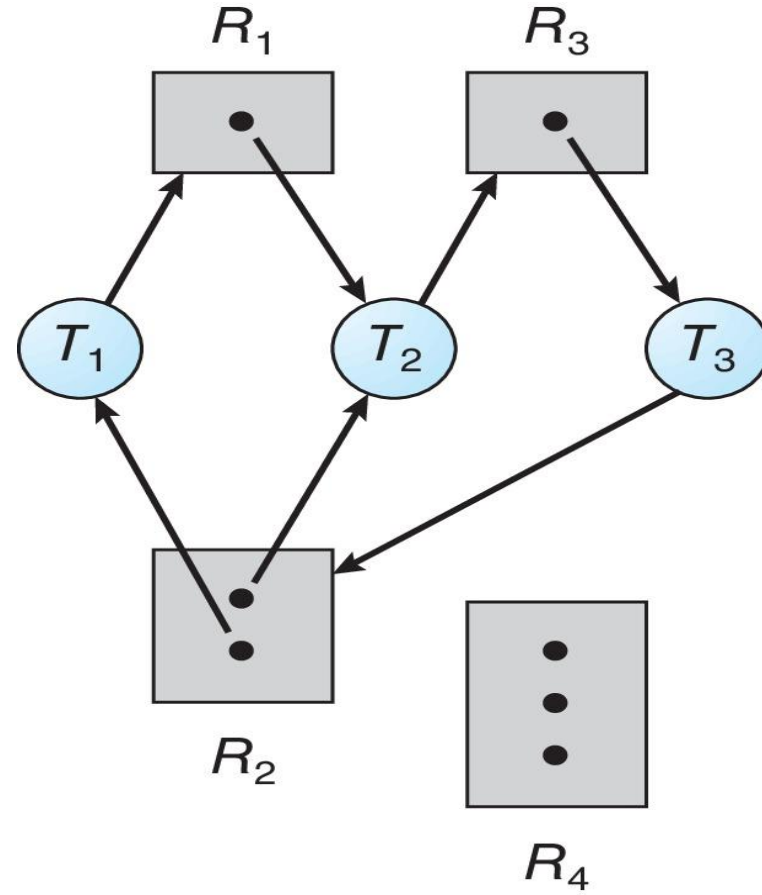
- V is partitioned into two types:
 - $P = \{P_1, P_2, \dots, P_n\}$, the set consisting of all the processes in the system
 - $R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all resource types in the system
- **request edge** – directed edge $P_i \rightarrow R_j$
- **assignment edge** – directed edge $R_j \rightarrow P_i$

Resource Allocation Graph Example

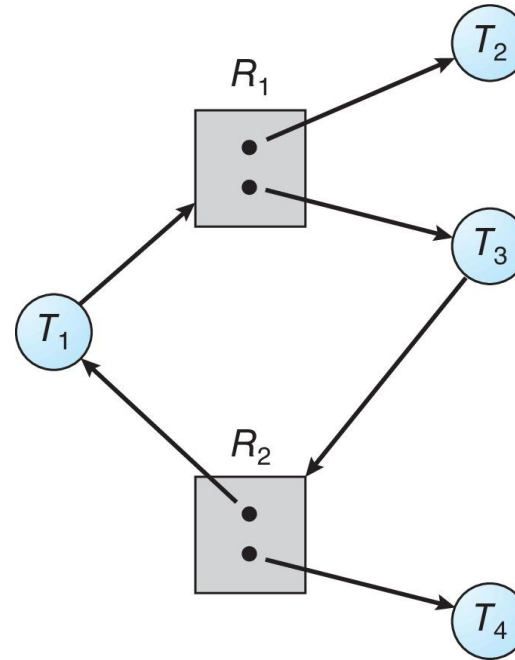
- One instance of R_1
- Two instances of R_2
- One instance of R_3
- Three instance of R_4
- T_1 holds one instance of R_2 and is waiting for an instance of R_1
- T_2 holds one instance of R_1 , one instance of R_2 , and is waiting for an instance of R_3
- T_3 is holds one instance of R_3



Resource Allocation Graph with a Deadlock



Graph with a Cycle But no Deadlock



Deadlock conclusion

- If graph contains no cycles $\Rightarrow$ no deadlock
- If graph contains a cycle $\Rightarrow$
 - If only one instance per resource type, then deadlock
 - If several instances per resource type, possibility of deadlock

Methods for Handling Deadlocks

- Deadlock prevention
- Deadlock avoidance

Deadlock Prevention

Invalidate one of the four necessary conditions for deadlock:

- **Mutual Exclusion** – not required for sharable resources (e.g., read-only files); must hold for non-sharable resources
- **Hold and Wait** – must guarantee that whenever a process requests a resource, it does not hold any other resources
 - Require processes to request and be allocated all its resources before it begins execution, or
 - Allow processes to request resources only when the process has none allocated to it.
 - Low resource utilization; starvation possible

Deadlock Prevention (Cont.)

- **No Preemption:**

- If a process that is holding some resources requests another resource that cannot be immediately allocated to it, then all the resources currently being held by the process are released
- Preempted resources are added to the list of resources for which the process is waiting
- Process will be restarted only when it can regain its old resources, as well as the new ones that it is requesting

- **Circular Wait:**

- Impose a total ordering of all resource types, and require that each process requests resources in an increasing order of enumeration

Deadlock Avoidance

Requires that the system has some additional *a priori* information available

- Simplest and most useful model requires that each process declare the ***maximum number*** of resources of each type that it may need
- The deadlock-avoidance algorithm dynamically examines the resource-allocation state to ensure that there can never be a circular-wait condition
- Resource-allocation *state* is defined by the number of available and allocated resources, and the maximum demands of the processes

Avoidance Algorithms

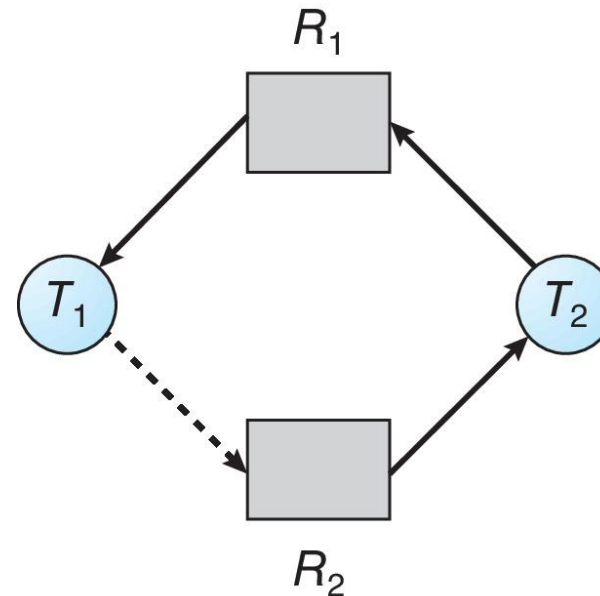
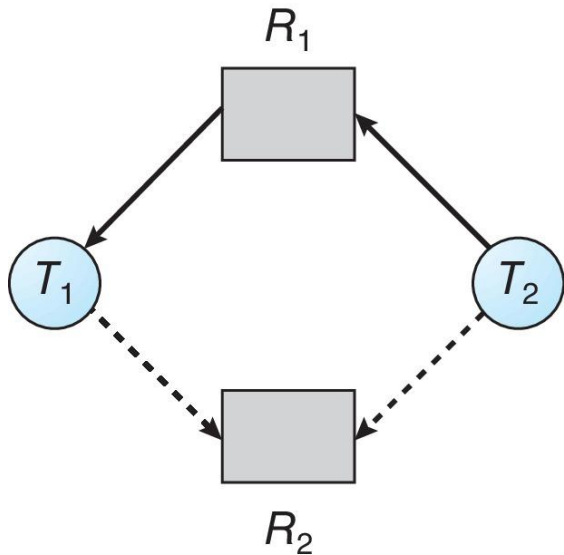
- Single instance of a resource type
 - Use a modified resource-allocation graph
- Multiple instances of a resource type
 - Use the Banker's Algorithm

Modified Resource Allocation Graph (RAG)

- **Claim edge** $P_i \dashrightarrow R_j$ indicates that process P_i may request resource R_j
- **Request edge** $P_i \rightarrow R_j$ indicates that process P_i requests resource R_j
 - Claim edge converts to request edge when a process requests a resource
- **Assignment edge** $R_j \rightarrow P_i$ indicates that resource R_j was allocated to process P_i
 - Request edge converts to an assignment edge when the resource is allocated to the process
- When a resource is released by a process, assignment edge reconverts to a claim edge
- Resources must be claimed *a priori* in the system

Resource-Allocation Graph Algorithm

- Suppose that process P_i requests a resource R_j
- The request can be granted only if converting the request edge to an assignment edge does not result in the formation of a cycle in the resource allocation graph



Banker's Algorithm

- Multiple instances of resources
- Each process must a priori claim maximum use
- When a process requests a resource, it may have to wait
- When a process gets all its resources it must return them in a finite amount of time

Data Structures for the Banker's Algorithm

Let n = number of processes, and m = number of resources types.

- **Available:** Vector of length m . If $available[j] = k$, there are k instances of resource type R_j available
- **Max:** $n \times m$ matrix. If $Max[i,j] = k$, then process P_i may request at most k instances of resource type R_j
- **Allocation:** $n \times m$ matrix. If $Allocation[i,j] = k$ then P_i is currently allocated k instances of R_j
- **Need:** $n \times m$ matrix. If $Need[i,j] = k$, then P_i may need k more instances of R_j to complete its task

$$Need[i,j] = Max[i,j] - Allocation[i,j]$$

Safety Algorithm / Banker's Algorithm

1. Let **Work** and **Finish** be vectors of length m and n , respectively. Initialize:

Work = **Available**

Finish [i] = **false** for $i = 0, 1, \dots, n-1$

2. Find an i such that both:

(a) **Finish** [i] = **false**

(b) **Need** _{i} ≤ **Work**

If no such i exists, go to step 4

3. **Work** = **Work** + **Allocation** _{i}
Finish [i] = **true**
go to step 2

4. If **Finish** [i] == **true** for all i , then the system is in a safe state

Example of Banker's Algorithm

- 5 processes P_0 through P_4 ;
3 resource types:
A (10 instances), B (5 instances), and C (7 instances)
- Snapshot at time T_0 :

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2
P_1	2 0 0	3 2 2	
P_2	3 0 2	9 0 2	
P_3	2 1 1	2 2 2	
P_4	0 0 2	4 3 3	

Example (Cont.)

- The content of the matrix ***Need*** is defined to be ***Max – Allocation***

	<u><i>Need</i></u>		
	<i>A</i>	<i>B</i>	<i>C</i>
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

Example (Cont.)

- Snapshot at time T_0 :

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>	<u>Need</u>
	<i>A B C</i>	<i>A B C</i>	<i>A B C</i>	<i>A B C</i>
P_0	0 1 0	7 5 3	3 3 2	7 4 3
P_1	2 0 0	3 2 2		1 2 2
P_2	3 0 2	9 0 2		6 0 0
P_3	2 1 1	2 2 2		0 1 1
P_4	0 0 2	4 3 3		4 3 1

- The system is in a safe state since the sequence $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ satisfies safety criteria

Example: P_1 Request (1,0,2)

- Check that Request $\leq$ Available (that is, $(1,0,2) \leq (3,3,2) \Rightarrow$ true

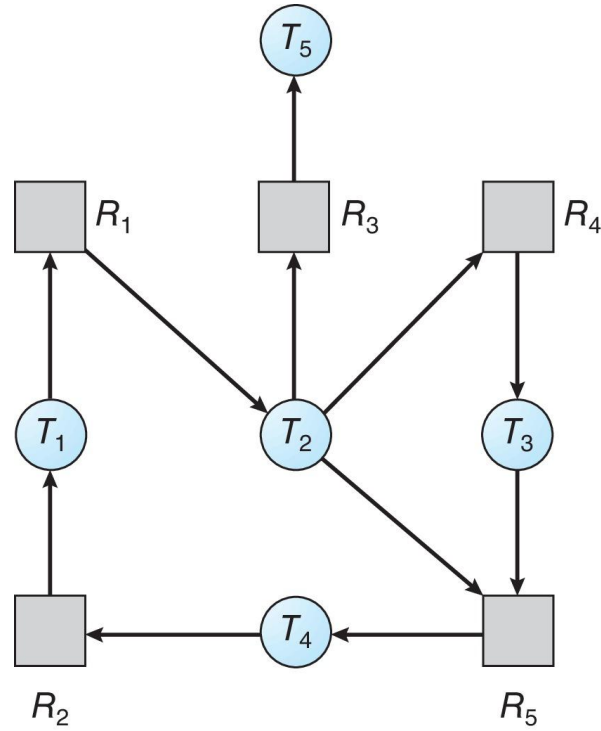
	<u>Allocation</u>	<u>Need</u>	<u>Available</u>
	A B C	A B C	A B C
P_0	0 1 0	7 4 3	2 3 0
P_1	3 0 2	0 2 0	
P_2	3 0 2	6 0 0	
P_3	2 1 1	0 1 1	
P_4	0 0 2	4 3 1	

- Executing safety algorithm shows that sequence $\langle P_1, P_3, P_4, P_0, P_2 \rangle$ satisfies safety requirement
- Can request for (3,3,0) by P_4 be granted?
- Can request for (0,2,0) by P_0 be granted?

Deadlock Detection

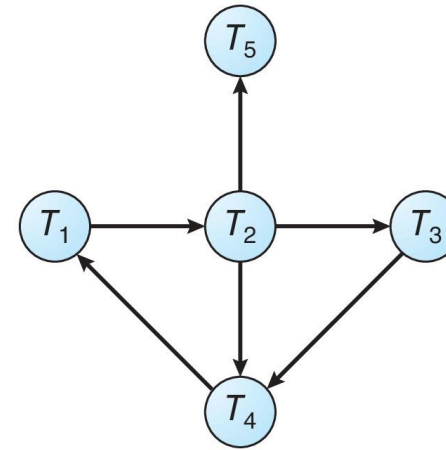
- Allow system to enter deadlock state
- Detection algorithm
- Recovery scheme

Resource-Allocation Graph and Wait-for Graph



(a)

Resource-Allocation Graph



(b)

Corresponding wait-for graph

Recovery from Deadlock: Process Termination

- Abort all deadlocked processes
- Abort one process at a time until the deadlock cycle is eliminated
- In which order should we choose to abort?
 - Priority of the process
 - How long process has computed, and how much longer to completion
 - Resources the process has used
 - Resources process needs to complete
 - How many processes will need to be terminated
 - Is process interactive or batch?

Recovery from Deadlock: Resource Preemption

- **Selecting a victim** – minimize cost
- **Rollback** – return to some safe state, restart process for that state
- **Starvation** – same process may always be picked as victim, include number of rollback in cost factor

Example of Banker's Algorithm

- 5 processes P_0 through P_4 ;
3 resource types:
A (10 instances), B (5 instances), and C (7 instances)
- Snapshot at time T_0 :

	<u>Allocation</u>	<u>Max</u>	<u>Available</u>	<u>Need</u>
	A B C	A B C	A B C	A B C
P_0	0 1 0	7 5 3	3 3 2	7 4 3
P_1	2 0 0	3 2 2		1 2 2
P_2	3 0 2	9 0 2		6 0 0
P_3	2 1 1	2 2 2		0 1 1
P_4	0 0 2	4 3 3		4 3 1

Example (Cont.)

- The content of the matrix ***Need*** is defined to be ***Max – Allocation***

	<u><i>Need</i></u>		
	<i>A</i>	<i>B</i>	<i>C</i>
P_0	7	4	3
P_1	1	2	2
P_2	6	0	0
P_3	0	1	1
P_4	4	3	1

- The system is in a safe state since the sequence $\langle P_1, P_3, P_4, P_2, P_0 \rangle$ satisfies safety criteria

Monitors

- Monitors are a high-level synchronization mechanism
- simplify process and thread synchronization
- Built on top of locks and are mostly used in multithreading systems like Java.
- Unlike semaphores, where the programmer must explicitly call **wait()** and **signal()**, monitors combine shared data and the operations on that data inside a single structure, making synchronization safer and easier to manage.

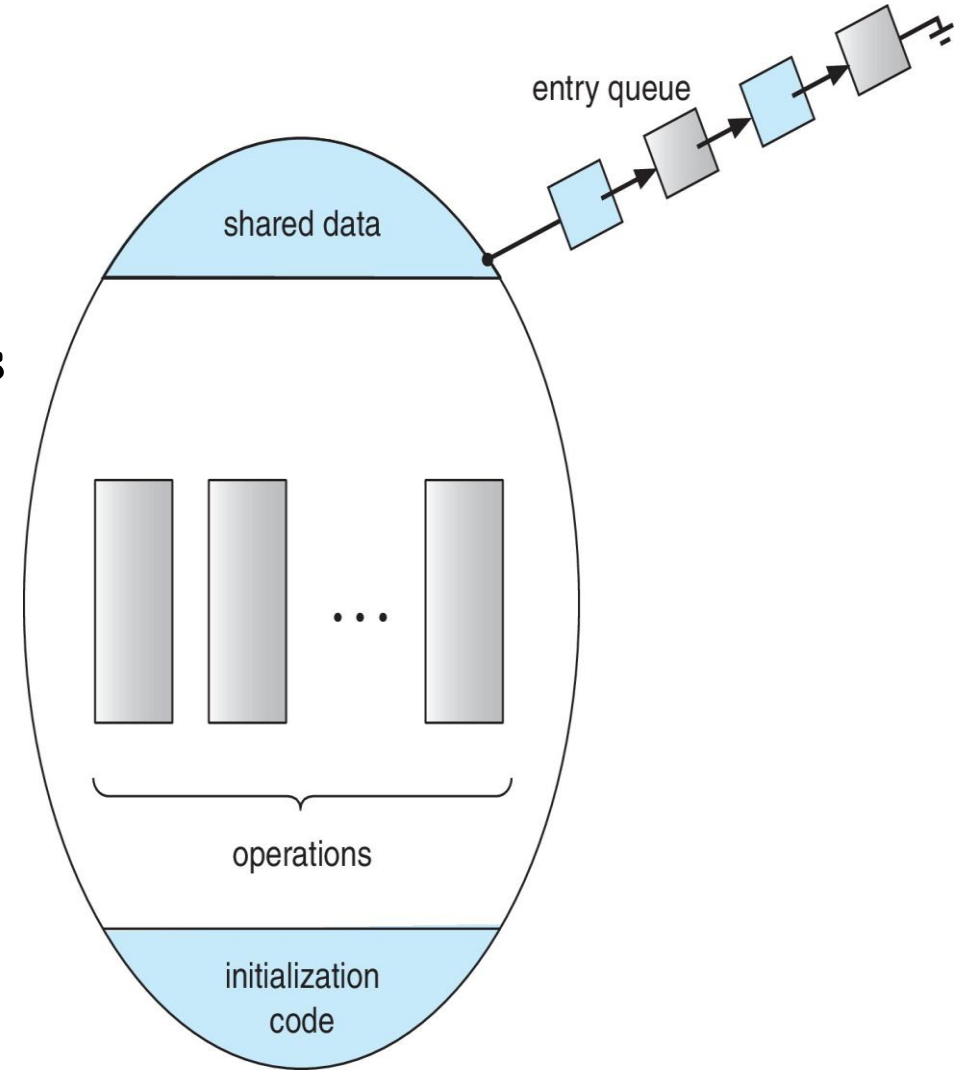
Monitors

```
monitor monitor-name
{
    // shared variable declarations
    function P1 (...) { ... }

    function P2 (...) { ... }

    function Pn (...) {.....}

    initialization code (...) { ... }
}
```



Liveness

- Processes may have to wait indefinitely while trying to acquire a synchronization tool such as a mutex lock or semaphore.
- Waiting indefinitely violates the progress and bounded-waiting criteria discussed at the beginning of this chapter.
- **Liveness** refers to a set of properties that a system must satisfy to ensure processes make progress.
- Indefinite waiting is an example of a liveness failure.